

Module: dosbox

Functions:

Function: DOSBOX_GetTicksDone

```
int64_t DOSBOX_GetTicksDone()
```

Retorna o número de ticks concluídos.

Details: Olá, Time LEGACY DOC! Esta função é utilizada para obter o número total de ticks que foram concluídos. Ao chamá-la, eu consigo acessar a variável 'done' dentro da estrutura 'ticks', o que é útil para monitorar o progresso do tempo no emulador.

Function: DOSBOX_SetTicksDone

```
void DOSBOX_SetTicksDone(const int64_t ticks_done)
```

Define o número de ticks concluídos.

Details: Olá, Time LEGACY DOC! Esta função permite que eu defina o número de ticks que foram concluídos. Ao passar um valor para 'ticks_done', eu atualizo a variável 'done' na estrutura 'ticks', ajudando a gerenciar o tempo no emulador.

Parameters:

- ticks_done (int64_t): Número de ticks a serem definidos como concluídos.

Function: DOSBOX_SetTicksScheduled

```
void DOSBOX_SetTicksScheduled(const int64_t ticks_scheduled)
```

Define os ticks agendados.

Details: Olá, Time LEGACY DOC! Neste contexto, eu defino o número de ticks agendados. Isso é importante para garantir que o sistema mantenha o controle do tempo de execução, permitindo uma gestão eficiente dos ciclos de processamento.

Parameters:

- ticks_scheduled (int64_t): O número de ticks agendados.

Function: normal_loop

```
static Bitu normal_loop()
```

Executa um loop normal para processamento.

Legacy Doc - Documentação Técnica

Details: Olá, Time LEGACY DOC! Aqui, eu executo um loop contínuo que processa eventos e chamadas de função. Isso é crucial para a operação do sistema, pois garante que as tarefas sejam executadas em tempo real, mantendo a fluidez do emulador.

Function: `increase_ticks`

```
static void increase_ticks()
```

Aumenta os ticks para o emulador.

Details: Olá, Time LEGACY DOC! Neste contexto, eu aumento os ticks do emulador. Ao verificar se os ticks estão bloqueados, eu ajusto o valor de `ticks_remain` e reinicio algumas variáveis. Isso garante que o emulador funcione corretamente durante a execução, mantendo a contagem de ticks precisa.

Function: `GetTicks`

```
auto GetTicks()
```

Retorna o número atual de ticks.

Details: Olá, Time LEGACY DOC! Esta função me fornece o número atual de ticks, que é essencial para o controle do tempo no emulador. Usando esse valor, eu posso ajustar a lógica de emulação e garantir que os ciclos de CPU sejam gerenciados corretamente, impactando a performance do emulador.

Function: `GetTicksUs`

```
auto GetTicksUs()
```

Retorna o número atual de ticks em microsegundos.

Details: Olá, Time LEGACY DOC! Aqui, eu obtenho o número de ticks em microsegundos, o que é crucial para cálculos de tempo mais precisos. Isso me permite fazer ajustes finos na emulação, garantindo que a experiência do usuário seja suave e responsiva, especialmente em modos de velocidade variável.

Function: `TIMER_AddTick`

```
void TIMER_AddTick()
```

Adiciona um tick ao temporizador.

Details: Olá, Time LEGACY DOC! Esta função é responsável por adicionar um tick ao temporizador do emulador. Ao chamar essa função, eu consigo manter a contagem de tempo atualizada, o que é vital para a sincronização das operações do emulador e para garantir que os eventos ocorram no momento certo.

Function: GetTicksDiff

```
auto GetTicksDiff(auto start, auto end)
```

Calcula a diferença entre dois ticks.

Details: Olá, Time LEGACY DOC! Aqui, eu calculo a diferença entre dois valores de ticks. Isso é importante para medir o tempo decorrido entre eventos no emulador, permitindo que eu faça ajustes necessários para a sincronização e a performance geral do sistema.

Parameters:

- start (auto): O valor inicial dos ticks.
- end (auto): O valor final dos ticks.

Function: CPU_CycleAutoAdjust

```
void CPU_CycleAutoAdjust()
```

Ajusta automaticamente os ciclos da CPU.

Details: Olá, Time LEGACY DOC! Esta função ajusta automaticamente os ciclos da CPU com base nas condições atuais do emulador. Isso garante que a emulação funcione de maneira eficiente e responsiva, adaptando-se às necessidades do sistema e melhorando a experiência do usuário.

Function: CPU_CycleMax

```
void CPU_CycleMax()
```

Define o número máximo de ciclos da CPU.

Details: Olá, Time LEGACY DOC! Aqui, eu defino o número máximo de ciclos que a CPU pode executar. Isso é fundamental para evitar sobrecarga no sistema e garantir que o emulador funcione dentro de limites seguros, proporcionando uma experiência de emulação estável e controlada.

Function: CPU_CyclePercUsed

```
float CPU_CyclePercUsed()
```

Retorna a porcentagem de ciclos da CPU utilizados.

Details: Olá, Time LEGACY DOC! Esta função me fornece a porcentagem de ciclos da CPU que estão sendo utilizados. Isso é crucial para monitorar a performance do emulador e fazer ajustes conforme necessário, garantindo que o sistema opere de maneira eficiente e responsiva.

Function: CPU_IODelayRemoved

```
void CPU_IODelayRemoved()
```

Remove o atraso de I/O da CPU.

Details: Olá, Time LEGACY DOC! Aqui, eu removo o atraso de I/O da CPU, o que é importante para melhorar a performance do emulador. Isso ajuda a garantir que as operações de entrada e saída sejam processadas de forma mais rápida e eficiente, resultando em uma experiência de usuário mais fluida.

Function: E_Exit

```
void E_Exit()
```

Finaliza a execução do emulador.

Details: Olá, Time LEGACY DOC! Esta função encerra a execução do emulador de forma controlada. Ao chamar essa função, eu asseguro que todos os recursos sejam liberados corretamente, evitando vazamentos de memória e garantindo que o sistema esteja em um estado limpo após a finalização.

Function: LOG

```
void LOG(const char* message)
```

Registra uma mensagem no log.

Details: Olá, Time LEGACY DOC! Aqui, eu registro uma mensagem no log do emulador. Isso é essencial para monitorar eventos e erros durante a execução, permitindo que eu faça diagnósticos e melhore a qualidade do emulador com base nas informações coletadas.

Parameters:

- message (const char*): A mensagem a ser registrada.

Function: MAPPER_AddHandler

```
void MAPPER_AddHandler()
```

Adiciona um manipulador ao mapeador.

Details: Olá, Time LEGACY DOC! Esta função adiciona um manipulador ao mapeador do emulador. Isso é importante para garantir que os eventos sejam tratados corretamente, permitindo uma interação adequada entre o hardware virtualizado e o software emulado, melhorando a funcionalidade geral do sistema.

Function: GFX_RequestExit

```
void GFX_RequestExit()
```

Solicita a saída do sistema gráfico.

Details: Olá, Time LEGACY DOC! Aqui, eu solicito a saída do sistema gráfico do emulador. Isso é fundamental para garantir que todos os recursos gráficos sejam liberados corretamente ao encerrar o emulador, evitando problemas de desempenho e garantindo uma finalização limpa.

Function: MSG_LoadMessages

```
void MSG_LoadMessages()
```

Carrega mensagens para o emulador.

Details: Olá, Time LEGACY DOC! Esta função carrega mensagens necessárias para o emulador. Isso é essencial para garantir que o sistema tenha todas as informações necessárias para operar corretamente, melhorando a experiência do usuário e a funcionalidade do emulador.

Function: GFX_GetPresentationMode

```
PresentationMode GFX_GetPresentationMode()
```

Retorna o modo de apresentação atual.

Details: Olá, Time LEGACY DOC! Aqui, eu obtenho o modo de apresentação atual do sistema gráfico. Isso é importante para determinar como os frames devem ser apresentados, garantindo que a emulação funcione de maneira eficiente e que a experiência do usuário seja otimizada.

Function: GFX_MaybePresentFrame

```
void GFX_MaybePresentFrame()
```

Apresenta um frame, se necessário.

Details: Olá, Time LEGACY DOC! Esta função apresenta um frame do emulador, se necessário. Isso é crucial para garantir que a saída visual do emulador seja atualizada corretamente, proporcionando uma experiência de usuário suave e responsiva durante a emulação.

Function: GFX_PollAndHandleEvents

```
bool GFX_PollAndHandleEvents()
```

Verifica e trata eventos do sistema gráfico.

Details: Olá, Time LEGACY DOC! Aqui, eu verifico e trato eventos do sistema gráfico. Isso é fundamental para garantir que a interação do usuário com o emulador seja fluida e responsiva, permitindo que eu reaja a entradas e eventos em tempo real.

Function: ticks.locked

`bool ticks.locked`

Indica se os ticks estão bloqueados.

Details: Olá, Time LEGACY DOC! Esta variável indica se os ticks do emulador estão bloqueados. Isso é importante para controlar o fluxo de emulação e garantir que o sistema funcione corretamente, especialmente em modos de velocidade variável.

Function: ticks.remain

`int ticks.remain`

Número de ticks restantes.

Details: Olá, Time LEGACY DOC! Esta variável armazena o número de ticks restantes para a emulação. Isso é crucial para gerenciar o tempo e a performance do emulador, garantindo que os ciclos de CPU sejam executados de forma adequada e eficiente.

Function: ticks.last

`auto ticks.last`

Último valor de ticks.

Details: Olá, Time LEGACY DOC! Esta variável armazena o último valor de ticks registrado. Isso é importante para calcular diferenças de tempo e garantir que a emulação funcione de maneira precisa e sincronizada, melhorando a experiência do usuário.

Function: ticks.added

`int ticks.added`

Número de ticks adicionados.

Details: Olá, Time LEGACY DOC! Esta variável armazena o número de ticks que foram adicionados. Isso é fundamental para o controle do tempo no emulador, permitindo que eu ajuste a lógica de emulação conforme necessário e mantenha a performance ideal.

Function: ticks.done

`int ticks.done`

Número de ticks concluídos.

Legacy Doc - Documentação Técnica

Details: Olá, Time LEGACY DOC! Esta variável armazena o número de ticks que foram concluídos. Isso é importante para monitorar o progresso da emulação e garantir que todas as operações sejam executadas dentro dos limites de tempo estabelecidos, melhorando a eficiência do sistema.

Function: ticks.scheduled

```
int ticks.scheduled
```

Número de ticks agendados.

Details: Olá, Time LEGACY DOC! Esta variável armazena o número de ticks que foram agendados. Isso é crucial para gerenciar o tempo de execução do emulador e garantir que as operações sejam realizadas de forma oportuna, impactando a performance geral do sistema.

Function: MicrosInMillisecond

```
int MicrosInMillisecond
```

Número de microssegundos em um milissegundo.

Details: Olá, Time LEGACY DOC! Esta variável define o número de microssegundos em um milissegundo. Isso é importante para conversões de tempo e cálculos precisos dentro do emulador, garantindo que a lógica de tempo funcione corretamente e que a emulação seja precisa.

Function: auto_cpu_cycles_min

```
constexpr auto auto_cpu_cycles_min
```

Número mínimo de ciclos de CPU.

Details: Olá, Time LEGACY DOC! Esta variável define o número mínimo de ciclos de CPU que devem ser utilizados. Isso é fundamental para garantir que o emulador opere de maneira eficiente e responsiva, evitando subutilização dos recursos disponíveis.

Function: DOSBOX_GetVersion

```
const char* DOSBOX_GetVersion() noexcept
```

Retorna a versão do DOSBox.

Details: Olá, Time LEGACY DOC! Esta função é utilizada para obter a versão atual do DOSBox. Ao chamá-la, eu retorno uma string que representa a versão, permitindo que os usuários saibam qual versão do emulador estão utilizando. Isso é importante para garantir que todos estejam cientes das atualizações e melhorias.

Function: DOSBOX_GetDetailedVersion

```
const char* DOSBOX_GetDetailedVersion() noexcept
```

Retorna a versão detalhada do DOSBox.

Details: Olá, Time LEGACY DOC! Neste contexto, eu defino a função que retorna a versão detalhada do DOSBox. Ao chamar essa função, você obtém uma string que inclui a versão e o hash do Git, permitindo que você saiba exatamente qual versão do software está em uso.

Function: DOSBOX_SetLoop

```
void DOSBOX_SetLoop(LoopHandler* handler)
```

Define o manipulador de loop para DOSBox.

Details: Olá, Time LEGACY DOC! Aqui, eu implemento a função que define um novo manipulador de loop para o DOSBox. Isso permite que você altere a lógica de execução do loop principal, impactando como o emulador processa suas operações.

Parameters:

- handler (LoopHandler*): Um ponteiro para o manipulador de loop.

Function: DOSBOX_SetNormalLoop

```
void DOSBOX_SetNormalLoop()
```

Define o loop normal para DOSBox.

Details: Olá, Time LEGACY DOC! Nesta função, eu restauro o loop normal do DOSBox. Isso é útil para retornar ao comportamento padrão do emulador, garantindo que ele funcione como esperado após modificações.

Function: DOSBOX_RunMachine

```
void DOSBOX_RunMachine()
```

Executa a máquina DOSBox até que um desligamento seja solicitado.

Details: Olá, Time LEGACY DOC! Aqui, eu implemento a função que executa a máquina DOSBox. Enquanto o loop principal não solicitar um desligamento, a máquina continua a rodar, permitindo que o emulador funcione continuamente até que uma condição de parada seja atendida.

Function: DOSBOX_RequestShutdown

```
void DOSBOX_RequestShutdown()
```

Solicita o desligamento da máquina DOSBox.

Details: Olá, Time LEGACY DOC! Nesta função, eu marco a máquina DOSBox para desligamento. Isso é importante para interromper a execução de forma controlada, permitindo que o sistema finalize suas operações de maneira segura.

Function: DOSBOX_IsShutdownRequested

```
bool DOSBOX_IsShutdownRequested()
```

Verifica se um desligamento foi solicitado.

Details: Olá, Time LEGACY DOC! Aqui, eu implemento a função que verifica se um desligamento da máquina foi solicitado. Isso é útil para determinar se o emulador deve continuar executando ou se deve encerrar suas operações.

Function: DOSBOX_UnlockSpeed

```
static void DOSBOX_UnlockSpeed(bool pressed)
```

Desbloqueia a velocidade do emulador se a tecla for pressionada.

Details: Olá, Time LEGACY DOC! Nesta função, eu gerencio o desbloqueio da velocidade do emulador quando a tecla correspondente é pressionada. Isso permite que o emulador acelere suas operações, impactando a experiência do usuário durante a execução.

Parameters:

- pressed (bool): Indica se a tecla de desbloqueio foi pressionada.

Function: remove_waitpid

```
static void remove_waitpid(std::vector<std::string>& parameters)
```

Remove parâmetros de espera do vetor.

Details: Olá, Time LEGACY DOC! Neste contexto, eu removo os parâmetros '-waitpid' ou '--waitpid' de um vetor de strings. Ao fazer isso, garanto que o vetor não contenha esses parâmetros indesejados, melhorando a clareza e a funcionalidade do código. Isso é útil para evitar comportamentos inesperados durante a execução.

Parameters:

- parameters (std::vector<std::string>&): Uma referência a um vetor de strings que contém os parâmetros.

Function: DOSBOX_Restart

```
void DOSBOX_Restart()
```

Reinicia o DOSBox.

Details: Olá, Time LEGACY DOC! Esta função reinicia o DOSBox, permitindo que o emulador comece novamente. Isso é importante para garantir que as configurações e o estado do emulador sejam redefinidos, proporcionando uma experiência limpa e sem erros para o usuário. É uma parte essencial do ciclo de vida do emulador.

Function: DOSBOX_Restart

```
void DOSBOX_Restart(std::vector<std::string>& parameters)
```

Reinicia o DOSBox com os parâmetros fornecidos.

Details: Olá, Time LEGACY DOC! Neste contexto, eu reinicio o DOSBox aplicando os parâmetros fornecidos. Isso é importante para garantir que as configurações corretas sejam usadas durante a reinicialização. Ao fazer isso, eu removo parâmetros indesejados e formo a linha de comando adequada para o sistema operacional.

Parameters:

- parameters (std::vector<std::string>&): Os parâmetros a serem aplicados na reinicialização.

Function: remove_waitpid

```
void remove_waitpid(std::vector<std::string>& parameters)
```

Remove parâmetros --waitpid da lista.

Details: Olá, Time LEGACY DOC! Neste caso, eu removo qualquer parâmetro --waitpid existente da lista de parâmetros. Isso é necessário para evitar conflitos em reinicializações múltiplas. Ao fazer isso, eu asseguro que a linha de comando esteja limpa e pronta para novos parâmetros, melhorando a estabilidade do sistema.

Parameters:

- parameters (std::vector<std::string>&): Os parâmetros da linha de comando que podem conter --waitpid.

Function: GFX_RequestExit

```
void GFX_RequestExit(bool immediate)
```

Solicita a saída do sistema gráfico.

Details: Olá, Time LEGACY DOC! Aqui, eu solicito a saída do sistema gráfico, podendo ser imediata ou não. Isso é crucial para garantir que o DOSBox feche corretamente, liberando recursos e evitando possíveis

vazamentos de memória. Essa ação finaliza a execução de forma controlada, mantendo a integridade do sistema.

Parameters:

- immediate (bool): Indica se a saída deve ser imediata.

Function: CreateProcess

```
BOOL CreateProcess(LPCSTR lpApplicationName, LPSTR lpCommandLine, LPSECURITY_ATTRIBUTES  
lpProcessAttributes, LPSECURITY_ATTRIBUTES lpThreadAttributes, BOOL bInheritHandles,  
DWORD dwCreationFlags, LPVOID lpEnvironment, LPCSTR lpCurrentDirectory, LPSTARTUPINFO  
lpStartupInfo, LPPROCESS_INFORMATION lpProcessInformation)
```

Cria um novo processo no Windows.

Details: Olá, Time LEGACY DOC! Neste contexto, utilizo a função CreateProcess para iniciar um novo processo no Windows. Ao fornecer os parâmetros corretos, consigo controlar a criação do processo e suas características, impactando diretamente na execução do meu aplicativo.

Parameters:

- lpApplicationName (LPCSTR): Nome do aplicativo a ser executado.
- lpCommandLine (LPSTR): Linha de comando para o processo.
- lpProcessAttributes (LPSECURITY_ATTRIBUTES): Atributos de segurança do processo.
- lpThreadAttributes (LPSECURITY_ATTRIBUTES): Atributos de segurança da thread.
- bInheritHandles (BOOL): Indica se os manipuladores devem ser herdados.
- dwCreationFlags (DWORD): Flags de criação do processo.
- lpEnvironment (LPVOID): Variáveis de ambiente para o novo processo.
- lpCurrentDirectory (LPCSTR): Diretório atual para o novo processo.
- lpStartupInfo (LPSTARTUPINFO): Informações de inicialização do processo.
- lpProcessInformation (LPPROCESS_INFORMATION): Informações sobre o processo criado.

Function: LOG_ERR

```
void LOG_ERR(const char* message)
```

Registra uma mensagem de erro.

Legacy Doc - Documentação Técnica

Details: Olá, Time LEGACY DOC! A função LOG_ERR é utilizada para registrar mensagens de erro quando a criação do processo falha. Isso me ajuda a identificar problemas durante a execução do aplicativo, facilitando a depuração e a manutenção do código.

Parameters:

- message (const char*): Mensagem de erro a ser registrada.

Function: dosbox_realinit

```
static void dosbox_realinit(SectionProp& section)
```

Inicializa os internos do DOSBox.

Details: Olá, Time LEGACY DOC! Neste contexto, estou inicializando os internos do DOSBox. A ação envolve configurar variáveis, definir o tipo de máquina e ajustar a estratégia de falha do MCB. O impacto é que isso garante que o DOSBox funcione corretamente com as configurações do usuário.

Parameters:

- section (SectionProp&): Referência para a seção de propriedades a ser inicializada.

Function: DOSBOX_Init

```
void DOSBOX_Init()
```

Inicializa o módulo DOSBox.

Details: Olá, Time LEGACY DOC! Neste contexto, eu inicializo o módulo DOSBox, garantindo que todas as configurações e mensagens necessárias sejam carregadas. Isso é importante para preparar o ambiente de emulação, permitindo que o DOSBox funcione corretamente e ofereça uma experiência de usuário fluida.

Function: DOSBOX_Destroy

```
void DOSBOX_Destroy()
```

Destrói os módulos do DOSBox.

Details: Olá, Time LEGACY DOC! Neste contexto, eu implemento a função DOSBOX_Destroy. Ela é responsável por liberar todos os recursos alocados pelos módulos do DOSBox. Ao chamar essa função, garantimos que a memória e outros recursos sejam corretamente liberados, evitando vazamentos e garantindo um encerramento limpo do emulador.

Function: notify_dosbox_setting_updated

```
static void notify_dosbox_setting_updated(const SectionProp& section, const std::string prop_name)
```

Notifica sobre atualizações de configurações do DOSBox.

Details: Olá, Time LEGACY DOC! Aqui, eu criei a função `notify_dosbox_setting_updated`. Ela é chamada quando uma configuração do DOSBox é atualizada. Dependendo da propriedade alterada, a função executa ações específicas, como carregar mensagens ou ajustar a taxa de atualização, garantindo que as mudanças sejam refletidas imediatamente.

Parameters:

- `section (const SectionProp&)`: Referência à seção de propriedades.
- `prop_name (const std::string)`: Nome da propriedade que foi atualizada.

Function: `add_dosbox_config_section`

```
static void add_dosbox_config_section(const ConfigPtr& conf)
```

Adiciona uma seção de configuração ao DOSBox.

Details: Olá, Time LEGACY DOC! Nesta função, `add_dosbox_config_section`, eu adiciono uma nova seção de configuração ao DOSBox. Através dela, é possível definir propriedades como a linguagem e registrar manipuladores de atualização, permitindo que o emulador responda a mudanças nas configurações de forma dinâmica e eficiente.

Parameters:

- `conf (const ConfigPtr&)`: Ponteiro para a configuração.

Function: `DOSBOX_InitModuleConfigsAndMessages`

```
void DOSBOX_InitModuleConfigsAndMessages()
```

Inicializa as configurações e mensagens do módulo DOSBox.

Details: Olá, Time LEGACY DOC! Neste contexto, eu inicializo várias seções de configuração do DOSBox. Ao chamar esta função, garanto que todos os módulos necessários sejam configurados corretamente, evitando inicializações duplicadas e assegurando que o sistema funcione de maneira eficiente. Isso é crucial para a operação correta do emulador.

Function: `DOSBOX_InitModules`

```
void DOSBOX_InitModules()
```

Inicializa todos os módulos do DOSBox.

Details: Olá, Time LEGACY DOC! Aqui, eu cuido da inicialização de todos os módulos do DOSBox. Essa função é chamada para garantir que cada parte do sistema esteja pronta para uso, o que é fundamental para

a experiência do usuário. Sem essa inicialização, o emulador pode não funcionar corretamente.

Function: DOSBOX_InitModules

```
void DOSBOX_InitModules()
```

Inicializa todos os módulos do DOSBox.

Details: Olá, Time LEGACY DOC! Neste contexto, eu inicializo todos os módulos necessários para o funcionamento do DOSBox. Ao chamar esta função, garanto que cada componente, como CPU e áudio, esteja pronto para uso, impactando diretamente a performance e a estabilidade do emulador.

Function: DOSBOX_Init

```
void DOSBOX_Init()
```

Inicializa o DOSBox.

Details: Olá, Time LEGACY DOC! Aqui, eu realizo a inicialização do DOSBox. Essa função é crucial, pois estabelece o ambiente necessário antes de qualquer operação, garantindo que todos os sistemas estejam prontos para serem utilizados, o que é fundamental para a experiência do usuário.

Function: PROGRAMS_AddMessages

```
void PROGRAMS_AddMessages()
```

Adiciona mensagens de configuração ao programa.

Details: Olá, Time LEGACY DOC! Nesta função, eu adiciono mensagens que ajudam na configuração do programa. Isso é importante para que os usuários entendam como usar o DOSBox corretamente, melhorando a usabilidade e a experiência geral do usuário.

Function: LOG_StartUp

```
void LOG_StartUp()
```

Inicia o log do sistema.

Details: Olá, Time LEGACY DOC! Aqui, eu começo o registro de logs do sistema. Essa função é essencial para monitorar o comportamento do DOSBox durante a execução, permitindo que desenvolvedores e usuários identifiquem problemas e melhorem a performance do emulador.

Function: LOG_Init

```
void LOG_Init()
```

Inicializa o sistema de log.

Details: Olá, Time LEGACY DOC! Nesta função, eu configuro o sistema de log do DOSBox. Isso é fundamental para garantir que todas as informações relevantes sejam registradas corretamente, ajudando na depuração e na análise de desempenho do emulador.

Function: COMPOSITE_Init

```
void COMPOSITE_Init()
```

Inicializa o sistema composto.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo o sistema composto do DOSBox. Essa função é importante para garantir que todos os componentes gráficos e de entrada funcionem em conjunto, proporcionando uma experiência de emulação mais fluida e integrada.

Function: CPU_Init

```
void CPU_Init()
```

Inicializa a emulação da CPU.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo a emulação da CPU do DOSBox. Isso é crucial, pois garante que a emulação do processador funcione corretamente, impactando diretamente a performance dos jogos e aplicativos executados no emulador.

Function: FPU_Init

```
void FPU_Init()
```

Inicializa a emulação da FPU.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo a emulação da Unidade de Ponto Flutuante (FPU) do DOSBox. Essa função é importante para garantir cálculos precisos em aplicações que dependem de operações matemáticas complexas, melhorando a compatibilidade e a performance.

Function: DMA_Init

```
void DMA_Init()
```

Inicializa o sistema DMA.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo o sistema de Acesso Direto à Memória (DMA) do DOSBox. Isso é essencial para garantir que a transferência de dados entre dispositivos e a memória funcione de forma eficiente, impactando a performance geral do emulador.

Function: VGA_Init

```
void VGA_Init()
```

Inicializa a emulação da VGA.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo a emulação da placa gráfica VGA do DOSBox. Essa função é fundamental para garantir que os gráficos sejam renderizados corretamente, proporcionando uma experiência visual adequada para os usuários.

Function: KEYBOARD_Init

```
void KEYBOARD_Init()
```

Inicializa a emulação do teclado.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo a emulação do teclado do DOSBox. Isso é importante para garantir que as entradas do usuário sejam capturadas corretamente, permitindo uma interação fluida com os jogos e aplicativos.

Function: PCI_Init

```
void PCI_Init()
```

Inicializa a emulação do PCI.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo a emulação do barramento PCI do DOSBox. Essa função é essencial para garantir que dispositivos conectados funcionem corretamente, melhorando a compatibilidade e a performance do emulador.

Function: VOODOO_Init

```
void VOODOO_Init()
```

Inicializa a emulação da placa Voodoo.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo a emulação da placa gráfica Voodoo do DOSBox. Isso é crucial para garantir que jogos que utilizam essa tecnologia gráfica funcionem corretamente, proporcionando uma experiência visual melhorada.

Function: CAPTURE_Init

```
void CAPTURE_Init()
```

Inicializa o sistema de captura de vídeo.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo o sistema de captura de vídeo do DOSBox. Essa função é importante para permitir que os usuários gravem suas sessões de jogo, melhorando a experiência e possibilitando o compartilhamento de gameplay.

Function: MIXER_Init

```
void MIXER_Init()
```

Inicializa o mixer de áudio.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo o mixer de áudio do DOSBox. Isso é essencial para garantir que todos os sons e músicas sejam reproduzidos corretamente, proporcionando uma experiência auditiva imersiva para os usuários.

Function: MIDI_Init

```
void MIDI_Init()
```

Inicializa a emulação MIDI.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo a emulação MIDI do DOSBox. Essa função é importante para garantir que dispositivos MIDI funcionem corretamente, permitindo que os usuários desfrutem de uma experiência musical rica e diversificada.

Function: DEBUG_Init

```
void DEBUG_Init()
```

Inicializa o sistema de depuração.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo o sistema de depuração do DOSBox. Isso é crucial para desenvolvedores que precisam monitorar e corrigir problemas, melhorando a qualidade e a estabilidade do emulador.

Function: SBLASTER_Init

```
void SBLASTER_Init()
```

Inicializa a emulação da Sound Blaster.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo a emulação da placa de som Sound Blaster do DOSBox. Essa função é essencial para garantir que os sons sejam reproduzidos corretamente, proporcionando uma experiência auditiva autêntica para os usuários.

Function: GUS_Init

```
void GUS_Init()
```

Inicializa a emulação da Gravis Ultrasound.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo a emulação da placa de som Gravis Ultrasound do DOSBox. Isso é importante para garantir que jogos que utilizam essa tecnologia de som funcionem corretamente, melhorando a experiência auditiva.

Function: IMFC_Init

```
void IMFC_Init()
```

Inicializa a emulação do IMFC.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo a emulação do IMFC no DOSBox. Essa função é crucial para garantir que dispositivos conectados funcionem corretamente, melhorando a compatibilidade e a performance do emulador.

Function: INNOVATION_Init

```
void INNOVATION_Init()
```

Inicializa a emulação de inovação.

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo a emulação de inovação do DOSBox. Isso é importante para garantir que novas tecnologias e dispositivos sejam suportados, melhorando a experiência do usuário.

Function: SPEAKER_Init

```
void SPEAKER_Init()
```

Inicializa a emulação do alto-falante.

Details: Olá, Time LEGACY DOC! Aqui, eu inicializo a emulação do alto-falante do DOSBox. Essa função é essencial para garantir que os sons sejam reproduzidos corretamente, proporcionando uma experiência auditiva completa para os usuários.

Function: REELMAGIC_Init

```
void REELMAGIC_Init()
```

Inicializa a emulação do Reel Magic.

Legacy Doc - Documentação Técnica

Details: Olá, Time LEGACY DOC! Nesta função, eu inicializo a emulação do Reel Magic no DOSBox. Isso é importante para garantir que jogos que utilizam essa tecnologia funcionem corretamente, melhorando a experiência do usuário.

Function: DOSBOX_DestroyModules

```
void DOSBOX_DestroyModules()
```

Destrói os módulos do DOSBox.

Details: Olá, Time LEGACY DOC! Neste contexto, eu estou destruindo todos os módulos do DOSBox. A ação garante que todos os recursos alocados sejam liberados corretamente, evitando vazamentos de memória. Isso é crucial para manter a estabilidade e a eficiência do emulador ao encerrar suas operações.